Performance Optimization of PicoRV32 RISC-V Processor Using Kogge-Stone Adder and Vedic Multiplier with Formal Verification

1st Dilip Chandra E
School of ECE
REVA University
Bengaluru, India
dilipchandra.e@reva.edu.in

2nd Jayasheel Vinay J
School of ECE
REVA University
Bengaluru, India
ugcet220791@reva.edu.in

3rd Jnanavi Nag
School of ECE
REVA University
Bengaluru, India
ugcet220367@reva.edu.in

4th Megharsha A Gowda
School of ECE
REVA University
Bengaluru, India
ugcet220478@reva.edu.in

5th Ramya R
School of ECE
REVA University
Bengaluru, India
22030137993@reva.edu.in

***Abstract*—RISC-V is gaining popularity as a free and open-source ISA. PicoRV32 is a small RISC-V soft core CPU designed to be deployed on FPGAs with the fewest hardware resource requirements. However, the arithmetic units of the core focus on hardware size requirements and can be a bottleneck when it comes to processing speed. This is a problem when implementing edge-AI style inference processing, as the primary operation is real-time processing dominated by loops of multiply-accumulate instructions with tight power and latency requirements. This paper proposes the design of an arithmetic unit core optimization using a 32-bit Kogge-Stone adder design and a Vedic multiplier using the Urdhva-Tiryakbhyam algorithm. The multiplier is integrated with the Pico Co-Processor Interface (PCPI) as a three-stage pipeline structure. Experimental results using the project dataset show a reduction in multiplication latency from 36 cycles to 3 cycles at 100 MHz operation, giving a 12.0× speedup factor. Additionally, the logic depth of the ALU add operation is reduced from 32 to 6 stages. Formal verification results show 0 failures from 70,598 simulation vectors with 13/13 GF(2) checks successful, validating the optimization design functionality. These results place the optimized core as a viable compute IP option for edge-AI SoCs with a need for efficient and verifiable integer arithmetic.**

***Index Terms*—PicoRV32, RISC-V, PCPI, Kogge-Stone adder, Vedic multiplier, edge AI, TinyML, hardware optimization, verification**

I. Introduction

The instruction set architecture of RISC-V has been well accepted in both academic and industrial designs due to its openness and flexibility [1]. PicoRV32 is a small RV32 core specifically targeted for FPGA implementation with low resource usage [2]. Although the small core is highly desirable for embedded systems, the arithmetic units in the baseline cores are primarily area-optimized, causing increased latency in arithmetic-intensive execution paths.

This is particularly relevant for edge-AI/TinyML workloads, which operate in close proximity to the sensor. In these cases, compact models still require a large number of integer-based multiply accumulate operations. In these systems, adder critical path and multiplication latency have a strong impact on inference latency as well as energy per inference. Always-on keyword spotting, anomaly detection in industrial sensoring, and low-resolution vision inference are a few examples of the relevant workloads where arithmetic acceleration is known to have a direct impact on system responsiveness and duty-cycled energy efficiency.

In order to mitigate the above limitations, this work proposes the optimization of the PicoRV32 arithmetic datapath. To achieve this, two hardware modifications have been proposed: the first is the integration of a 32-bit Kogge-Stone parallel prefix adder in place of the standard ripple-carry adder, and the second is the integration of a 32×32 Vedic multiplier-based PCPI accelerator.

This is because the proposed integration strategy based on the PCPI metric is designed as a modular approach, where the optimized architecture remains as a CPU core-level IP extension, allowing it to be plugged into various SoC scenarios (microcontroller, IoT sensing, or accelerative edge AI subsystems) without any changes to the pipeline.

The paper is prepared based on the existing project outputs as baseline evidence, including architecture, implementation, and results in IEEE conference style.

Contributions. The principal contributions of this work are:

- A 32-bit Kogge-Stone parallel-prefix adder integrated in the PicoRV32 ALU, reducing structural logic depth from 32 to 6 stages (5.3×).
- A modular three-stage pipelined Vedic (32×32) multiplier attached through the Pico Co-Processor Interface (PCPI), lowering MUL latency from 36 cycles to 3 cycles at 100 MHz (12.0×).

- A combined functional and algebraic verification flow reporting zero failures across 70,598 simulation vectors and 13/13 successful GF(2) checks for the optimized arithmetic datapath.
- A non-invasive integration strategy that preserves PicoRV32 instruction-level compatibility, positioning the optimized core as a reusable compute IP for edge-AI SoCs.

II. Related Work

A. PicoRV32 Architecture and Baseline Trade-offs

The PicoRV32 is a small soft core RISC-V processor with low FPGA resource utilization and architectural simplicity as design objectives [2], [3]. Its design generally prioritizes area efficiency over peak arithmetic throughput, which makes arithmetic-intensive instruction streams sensitive to ALU and multiplier latency. The RV32 user-level ISA context and compatibility constraints are well established in the RISC-V specification literature [1], [4].

B. Vedic Multipliers and Adder Co-design

Vedic multiplication using the technique of Urdhva-Tiryakbhyam has been explored as an alternative to traditional multiplication with the objective of minimizing latency in programmable logic-based designs [5], [6]. Various research works have also demonstrated the significant influence of the choice of internal adders on the latency, area, and power consumption of the multiplier circuit, with Han Carlson, modified CLA, and other variants showing significant improvements over traditional designs [7]–[9].

C. Parallel Prefix Kogge-Stone Addition

The Kogge-Stone adder is one of the fundamental parallel prefix structures, offering efficient carry computation through its structure, which has a logarithmic depth [10]. Modern research and comparative results verify its good delay performance compared to ripple-based counterparts, as well as its competitive nature compared to other efficient adder topologies for moderate or wide operand sizes [11], [12].

D. Algebraic and Formal Verification of Arithmetic Circuits

Formal verification of arithmetic circuits, specifically multipliers, has been performed through algebraic approaches, represented as polynomials, as part of the verification flow, together with traditional simulation-based methodologies [13]. Further research has verified its applicability to various scalable forms of arithmetic circuits, including finite field and multiplier circuits.

E. Comparison with Other RISC-V Optimization Techniques

Several open-source RISC-V cores adopt different strategies to improve arithmetic throughput. Rocket Chip [14] relies on an in-order pipeline augmented with optional FPU and configurable multiplier units; BOOM [15] pursues higher IPC through an out-of-order superscalar microarchitecture; and the SHAKTI family [16] provides parameterized DSP-style multipliers as part of the core. These approaches deliver strong performance but typically increase area, power, and verification complexity beyond what edge-AI/FPGA-class deployments can absorb. In contrast, the optimization proposed here keeps the PicoRV32 pipeline unchanged and confines the high-speed adder and pipelined Vedic multiplier behind the PCPI handshake, yielding a 12.0× multiplier speedup with a non-invasive, instruction-compatible drop-in path.

F. Relevance to This Work

The design direction taken in this work is an amalgamation of the well-established paths by incorporating the Kogge-Stone addition, Vedic multiplication, and correctness-driven validation techniques into a single optimization flow for the PicoRV32 processor. This paper attempts to assess the effect of the combined optimization techniques at the processor integration level with PCPI-based acceleration and result summarization, making it more relevant to edge-based IoT and TinyML deployments where integer arithmetic and determinism are critical constraints.

III. System Architecture of the Optimized PicoRV32

The optimized architecture enhances baseline PicoRV32 with two arithmetic modules while preserving processor-level control and memory interfaces.

The block diagram in Fig. 1 represents the architecture pointers, i.e., PicoRV32 core, 32-bit Kogge-Stone ALU adder, PCPI-based Vedic multiplier accelerator, register/control logic, and memory interface.

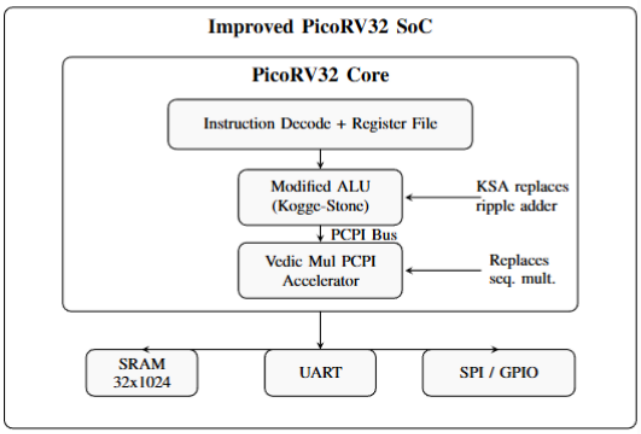

Fig. 1. Improved PicoRV32 SoC block diagram with Kogge-Stone ALU replacement, PCPI-based Vedic multiplier acceleration, and peripheral interfaces.

Fig. 1. Improved PicoRV32 SoC block diagram with Kogge-Stone ALU replacement, PCPI-based Vedic multiplier acceleration, and peripheral interfaces.

The first enhancement involves replacing the ripple carry addition in the arithmetic path with the Kogge-Stone prefix addition. The second enhancement involves the pipelined multiplier instead of the PCPI, allowing the multiplication

operations to be completed by the external accelerator logic within a fixed latency.

This architecture improves the arithmetic path and maintains the lightweight design approach for the PicoRV32.

IV. Hardware Implementation

A. Kogge-Stone Parallel Prefix Adder

The 32-bit Kogge-Stone adder calculates the per-bit generate and propagate values:

$$G_i = A_i \cdot B_i, \quad P_i = A_i \oplus B_i. \tag{1}$$

The adder uses a parallel prefix adder, which reduces the logic depth from linear to logarithmic time complexity. The logic depth can be represented as:

$$D_{\mathrm{KSA}} = \lceil \log_2 N \rceil + 1. \tag{2}$$

When $N = 32$, the logic depth is 6 stages, whereas the RCA would take 32 stages.

B. Vedic Multiplier Architecture

The Vedic multiplier uses a hierarchical Urdhva-Tiryakbhyam method from 2×2 blocks to 32×32 blocks to achieve a 64-bit output. The architecture is suitable for parallel partial product computation and addition.

C. PCPI Integration

The Vedic multiplier is integrated using the Pico Co-Processor Interface (PCPI) as a three-stage pipeline:

- Stage 1 - Operand Capture
- Stage 2 - Multiplication and Sign Processing
- Stage 3 - Registered Output/Handshake

The implementation was run at 100 MHz, which corresponds to about 3 clock cycles total latency for multiplication completion.

V. Combined Results and Discussion

This section summarizes the project results based on the obtained figures and CSV data.

A. Multiplication Latency Comparison

The baseline PCPI multiplication latency is a sequence of shift and add operations, which requires 36 cycles, whereas the optimized Vedic PCPI method requires only 3 cycles.

In Fig. 2, we can see the difference between the baseline and the optimized method in terms of cycles and actual time with a 100-MHz clock speed. It is also important to convert the time difference from cycles to actual time in terms of ns because firmware developers or SoC integrators usually prefer to reserve a certain amount of time in microseconds rather than cycles while developing firmware.

What is interesting is that not only is the optimization beneficial for the peak performance of the multiplication, but it is also beneficial for the predictability of the multiplication in terms of the time window available to perform the multiplication. A small amount of time is available to perform the multiplication operation, which is useful while scheduling the kernels with control or edge-AI functions repeatedly calling the MAC functions.

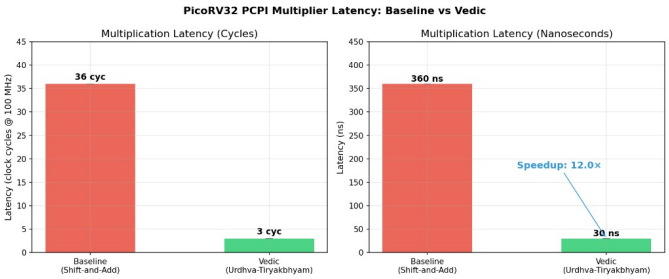

Fig. 2. PCPI multiplier latency comparison in cycles and nanoseconds.

B. Latency Distribution

For the 64 vectors, the baseline latency remains the same at 36 cycles, and the optimized latency also remains the same at 3 cycles.

From the latency distribution in Fig. 3 for the test case, it is clear that the implementations are deterministic. This is because there is no spread on the graph, which implies that the operand values have no impact on the timing in the code paths. The deterministic latency distribution for the implementations is as important as the average speedup for the implementations.

C. Functional Verification

The functional and formal verification have zero failures in the reported suites.

Fig. 4 shows a summary of the unit-level arithmetic tests, PCPI interaction tests, and algebraic verification. While there is a large number of passing tests, this provides us with confidence that the arithmetic acceleration is not inducing any regressions in the arithmetic functionality. This type of verification is important when we think about the placement of the design in edge-AI systems.

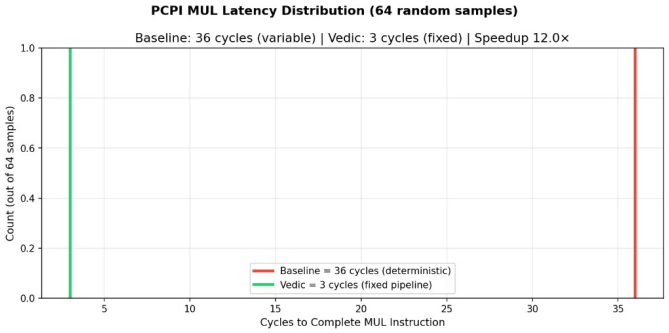

Fig. 3. PCPI latency distribution across sampled test vectors.

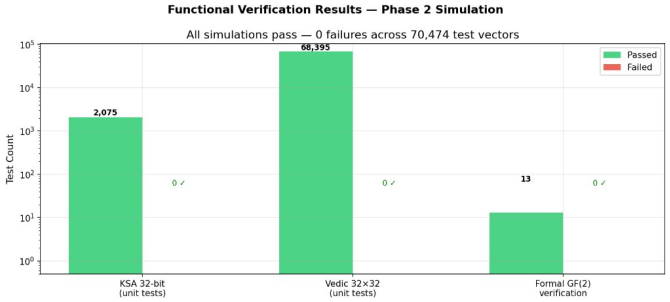

Fig. 4. Functional verification summary.

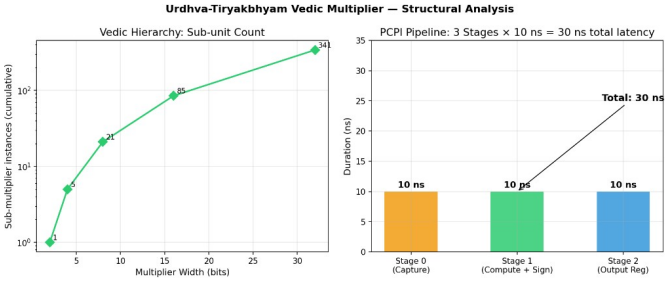

Fig. 6. Vedic multiplier hierarchy and PCPI pipeline analysis.

D. Adder Logic Depth Analysis

The Kogge-Stone adder offers a reduction in depth growth compared to ripple carry architecture.

Fig. 5 illustrates the growth rate of logic depth with respect to operand bit-width and demonstrates the logarithmic growth rate for KSA compared to a linear growth rate for RCA. The widening gap indicates that carry-propagation optimization grows more important with increasing bit-width.

Although this section refers to structural depth rather than timing depth, it offers a first-order explanation for achieving higher $F_{\max}$ with carry replacement.

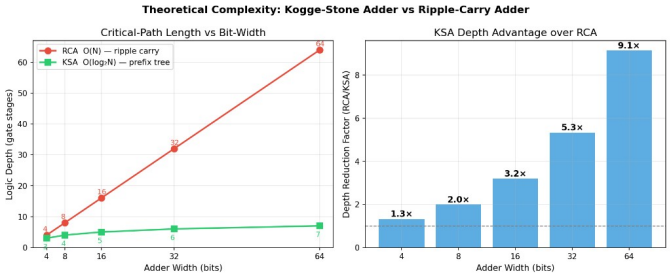

Fig. 5. Theoretical logic depth comparison for RCA versus KSA.

E. Adder Logic Depth Analysis

Kogge-Stone Adder reduces the depth growth rate compared to the ripple carry architecture.

In Fig. 5, we can see the growth rate of logic depth with respect to the bit-width of the operands and how it shows a logarithmic growth rate for the KSA compared to a linear growth rate for the RCA. This shows how the gap is widening, and therefore, the optimization of carry propagation becomes more significant as the bit-width grows.

Although this is about structural depth and not timing depth, it is a first-order explanation for achieving a greater $F_{\max}$ with carry replacement.

F. Overall Performance Summary

Table I offers a summary of the baseline-to-optimized performance.

The table and Fig. 7 together provide a compact representation of the trade-off. The most significant acceleration occurs in multiplication latency; however, the reduction in adder depth contributes to a more general timing acceleration of all arithmetic operations.

The dual representation of these results is helpful in relating them to end-application mapping. For example, an application with many loops dominated by multiplication will directly benefit from acceleration, while a more mixed computation and control workload will also benefit from the reduction in adder carry depth.

TABLE I
PERFORMANCE COMPARISON: BASELINE VS OPTIMIZED

Metric	Baseline	Optimized	Gain
PCPI MUL latency (cycles)	36	3	12.0×
PCPI MUL latency (ns @100MHz)	360	30	12.0×
KSA logic depth (32-bit)	32	6	5.3×
Adder critical path	$O(N)$	$O(\log_2 N)$	–
Multiplier completion	$O(N)$ cycles	$O(1)$ cycles	–

The measured 12.0× multiplier acceleration is the major performance boost reported by this work. This is also because the optimized path is integrated via PCPI rather than intrusive cores. This way, architectural risks are kept under control while maintaining instruction-level compatibility.

The replacement of KSA also improves adder logic depth. This should help in achieving timing closure margins for higher-frequency synthesis targets. Although this paper demonstrates this at a simulation and complexity level, FPGA post-place-and-route timing and area characterization can be included in future extensions.

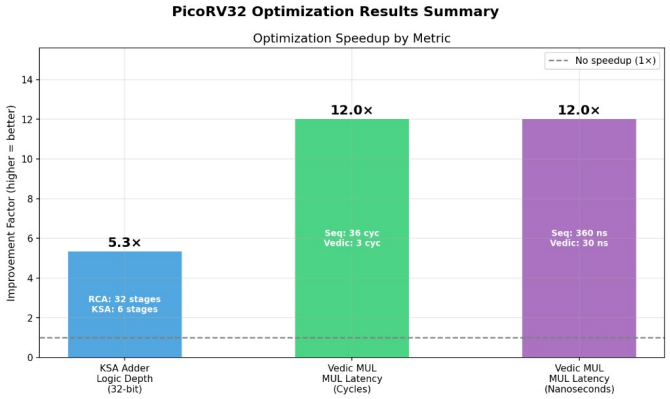

Fig. 7. Summary of measured optimization gains.

G. Scalability and Extensibility

The hierarchical Urdhva-Tiryakbhyam construction used for the multiplier scales naturally beyond 32 bits: a 64×64 datapath can be obtained by composing four 32×32 instances with a final reduction stage, and the same PCPI handshake remains valid, only the operand and result widths change. The Kogge-Stone adder likewise grows in depth as $\lceil \log_2 N \rceil + 1$, so wider operands keep the same logarithmic advantage over a ripple-carry baseline. For multi-core scenarios, each core can either embed a private PCPI accelerator – preserving the deterministic 3-cycle latency reported here – or share a single arbiter-fronted Vedic unit across cores when area is the dominant constraint. These extensions reuse the verified arithmetic blocks unchanged and therefore do not invalidate the formal-verification results presented above.

From an SoC application perspective, this arithmetic acceleration opportunity directly maps to edge AI software kernels with integer MAC-dominated execution profiles. Reducing multiply latency and improving adder depth can result in lower inference times at a given clock rate or equivalent throughput at lower clock and/or power operating points. Thus, this optimized PicoRV32 core can be a good candidate for a digital compute platform for SoCs that aim to integrate edge AI capabilities.

VI. Conclusion

The paper includes a study on the optimization of the PicoRV32 RISC-V processor's performance. In this study, a Kogge-Stone adder and a PCPI-based Vedic Multiplier have been used.

The study's results have demonstrated that the adder's logic depth is reduced from 32 levels to 6 levels for a 32-bit RISC-V processor. In the context of the Vedic Multiplier, the latency of the multiplication process is reduced from 36 cycles to 3 cycles at a frequency of 100 MHz. This shows a 12.0× improvement in the processor's performance. In the context of the study's verification, the results have demonstrated 0 failures among 70,598 simulation vectors. Moreover, the study's formal checks have demonstrated full success.

The study's results have demonstrated that the integration of high-speed arithmetic cores into lightweight open-source RISC-V cores is a viable option. In the context of the study's design direction, the results have demonstrated the viability of the design direction for the development of edge-AI-capable embedded SoCs. Future work will be conducted through the same PCPI design direction, with the development of quantized AI-friendly instructions.

References

[1] RISC-V International, "The RISC-V instruction set manual, volume i: Unprivileged ISA," https://riscv.org/technical/specifications/, 2024, accessed: Mar. 2026.

[2] C. Wolf, "PicoRV32: A size-optimized RISC-V CPU core," https://github.com/YosysHQ/picorv32, 2015, accessed: Mar. 2026.

[3] ——, "PicoRV32 – a size-optimized RISC-V CPU core," https://github.com/cliffordwolf/picorv32, 2015, accessed: Mar. 2026.

[4] A. Waterman and K. Asanović, "The RISC-V instruction set manual, volume i: User-level ISA, version 2.2," EECS Department, University of California, Berkeley, Tech. Rep., May 2017.

[5] A. Mehta and B. Vyas, "A survey on vedic multiplier architectures and FPGA implementations," *International Journal of Computer Applications*, vol. 95, no. 25, pp. 23–28, 2014.

[6] N. Sharma *et al.*, "An efficient digital vedic multiplier with an enhanced adder," in *Proc. IEEE Int. Conf. on Intelligent Computing and Control Systems (ICICCS)*, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10959443

[7] S. Kakde and P. Thakare, "Implementation of an efficient multiplier based on vedic mathematics," *International Journal of Innovative Science, Engineering and Technology (IJISET)*, vol. 1, no. 6, Aug. 2014. [Online]. Available: https://www.ijiset.com/v1s6/IJISET_V1_I6_19.pdf

[8] R. Anitha and V. Bagyaveereswaran, "High performance vedic multiplier using han-carlson adder," *International Journal of Engineering Research and Technology (IJERT)*, vol. 2, no. 11, Nov. 2013. [Online]. Available: https://www.ijert.org/high-performance-vedic-multiplier-using-han-carlson-adder

[9] M. Priya and K. Baskaran, "Optimized vedic multiplier using adder approximations and modified CLA," in *Proc. Int. Conf. on Computing and Communications Technologies (ICCCT)*, 2015. [Online]. Available: https://ir.psgitech.ac.in/id/eprint/930/

[10] P. M. Kogge and H. S. Stone, "A parallel algorithm for the efficient solution of a general class of recurrence equations," *IEEE Transactions on Computers*, vol. C-22, no. 8, pp. 786–793, 1973.

[11] S. Sundar and M. Chidambaram, "Design of high-speed and energy-efficient parallel prefix Kogge-Stone adder," *Juni Khyat*, vol. 9, no. 12, pp. 87–93, Dec. 2019. [Online]. Available: http://junikhyatjournal.in/dec19/87.pdf

[12] B. Parhami, "Kogge-Stone adder: A scalable design solution to high-speed addition," *ICTACT Journal on Microelectronics*, vol. 11, no. 3, pp. 2151–2156, Oct. 2015. [Online]. Available: https://ictactjournals.in/paper/IJME_Vol_11_Iss_3_Paper_2_2151_2156.pdf

[13] M. Ciesielski *et al.*, "Formal verification of Galois field multipliers using computer algebra techniques," in *Proc. 25th Int. Conf. on VLSI Design (VLSID)*, 2012, pp. 136–141. [Online]. Available: https://ieeexplore.ieee.org/document/6167783

[14] K. Asanović, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio *et al.*, "The rocket chip generator," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, Apr. 2016.

[15] C. Celio, P.-F. Chiu, B. Nikolić, D. A. Patterson, and K. Asanović, "BOOM v2: An open-source out-of-order RISC-V core," in *First Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2017.

[16] N. Gala, A. Menon, R. Bodduna, G. S. Madhusudan, and V. Kamakoti, "SHAKTI processors: An open-source hardware initiative," in *Proc. 29th Int. Conf. on VLSI Design (VLSID)*, 2016, pp. 7–8.